

Week 1, Day 3-4: Joins & Relationships

INNER JOIN, LEFT JOIN, Foreign Keys, Table Relationships

Table of Contents

1. [Understanding Relationships](#)
 2. [Foreign Keys](#)
 3. [INNER JOIN](#)
 4. [LEFT JOIN \(LEFT OUTER JOIN\)](#)
 5. [RIGHT JOIN \(RIGHT OUTER JOIN\)](#)
 6. [FULL OUTER JOIN](#)
 7. [CROSS JOIN](#)
 8. [Self Join](#)
 9. [Multiple Joins](#)
 10. [Practice Exercises](#)
 11. [Interview Q&A](#)
-

Understanding Relationships

What are Relationships?

Relationships connect data across multiple tables by linking related information using Primary Keys and Foreign Keys.

Why Do We Need Relationships?

- **Avoid Data Duplication:** Instead of repeating instructor info in every course, store it once
- **Data Integrity:** Ensure valid data by linking to existing records
- **Efficient Queries:** Combine data from multiple tables
- **Normalization:** Organize data efficiently

Real-World Example:

Without Relationships (Bad):

CourseID	CourseName	InstructorName
1	.NET Full Stack	Mr. Sharma
2	Web Dev	Ms. Priya
3	Database	Mr. Sharma

← Duplicate!

With Relationships (Good):

Courses Table:			Instructors Table:	
CID	Name	ID	ID	Name
1	.NET	1	1	Sharma
2	WebDev	2	2	Priya

```
| 3      | DB      | 1 |----> | 1 | Sharma|
+-----+-----+---+      +-----+
FK points to PK
```

Types of Relationships:

Type	Meaning	Example
1-to-Many	One parent has many children	1 Instructor teaches Many Courses
Many-to-Many	Many can relate to many	Many Students take Many Courses
1-to-1	One relates to exactly one	1 Person has 1 Passport

Foreign Keys

What is a Foreign Key?

A **Foreign Key (FK)** is a column in one table that references the Primary Key in another table. It enforces the relationship.

Syntax:

```
CREATE TABLE ChildTable (
    ChildID INT PRIMARY KEY,
    ChildName NVARCHAR(50),
    ParentID INT,
    FOREIGN KEY (ParentID) REFERENCES ParentTable(ParentID)
);
```

Example 1: Create Instructors Table

```
CREATE TABLE Instructors (
    InstructorID INT PRIMARY KEY IDENTITY(1,1),
    InstructorName NVARCHAR(100) NOT NULL,
    Email VARCHAR(100) UNIQUE,
    Department NVARCHAR(50),
    Salary DECIMAL(10,2)
);
```

Example 2: Create Courses Table with Foreign Key

```
CREATE TABLE Courses (
    CourseID INT PRIMARY KEY IDENTITY(1,1),
    CourseName NVARCHAR(100) NOT NULL,
    InstructorID INT NOT NULL,
    Credits INT CHECK (Credits > 0),
    Duration INT,
    FOREIGN KEY (InstructorID) REFERENCES Instructors(InstructorID)
);
```

Explanation: - InstructorID INT NOT NULL → Cannot be empty, must exist in Instructors table - FOREIGN KEY (InstructorID) → This column is a FK - REFERENCES Instructors(InstructorID) → Points to Instructors table's PK

Example 3: Create Students Table

```
CREATE TABLE Students (
```

```

StudentID INT PRIMARY KEY IDENTITY(1,1),
FirstName NVARCHAR(50) NOT NULL,
LastName NVARCHAR(50) NOT NULL,
Email VARCHAR(100) UNIQUE,
EnrollmentDate DATETIME DEFAULT GETDATE()
);

```

Example 4: Create Enrollments Table (Many-to-Many)

```

CREATE TABLE Enrollments (
    EnrollmentID INT PRIMARY KEY IDENTITY(1,1),
    StudentID INT NOT NULL,
    CourseID INT NOT NULL,
    EnrollmentDate DATETIME DEFAULT GETDATE(),
    Grade DECIMAL(3,2),
    FOREIGN KEY (StudentID) REFERENCES Students(StudentID),
    FOREIGN KEY (CourseID) REFERENCES Courses(CourseID)
);

```

Insert Sample Data:

```

-- Insert Instructors
INSERT INTO Instructors (InstructorName, Email, Department, Salary)
VALUES
('Mr. Sharma', 'sharma@email.com', 'IT', 80000),
('Ms. Priya', 'priya@email.com', 'IT', 75000),
('Mr. Kumar', 'kumar@email.com', 'IT', 70000);

-- Insert Courses
INSERT INTO Courses (CourseName, InstructorID, Credits, Duration)
VALUES
('.NET Full Stack', 1, 4, 12),
('Web Development', 2, 4, 10),
('Database Design', 1, 3, 8);

-- Insert Students
INSERT INTO Students (FirstName, LastName, Email) VALUES
('Raj', 'Kumar', 'raj@email.com'),
('Priya', 'Singh', 'priya.s@email.com'),
('Arjun', 'Patel', 'arjun@email.com'),
('Neha', 'Sharma', 'neha@email.com');

-- Insert Enrollments
INSERT INTO Enrollments (StudentID, CourseID, Grade) VALUES
(1, 1, 3.8), -- Raj enrolled in .NET
(1, 2, 3.7), -- Raj enrolled in Web Dev
(2, 1, 3.9), -- Priya enrolled in .NET
(3, 2, 3.5), -- Arjun enrolled in Web Dev
(4, 3, 3.6); -- Neha enrolled in Database Design

```

△ Foreign Key Constraints:

```

-- This will FAIL - InstructorID 99 doesn't exist
INSERT INTO Courses (CourseName, InstructorID, Credits)
VALUES (.NET Advanced', 99, 3);

-- This works - InstructorID 1 exists
INSERT INTO Courses (CourseName, InstructorID, Credits)
VALUES (.NET Advanced', 1, 3);

```

INNER JOIN

What is INNER JOIN?

INNER JOIN returns only rows that have matching values in BOTH tables.

Visual Representation:

Table A		Table B
+-----+		+-----+
1 x		1 a
2 y	n	3 c
3 z		4 d
+-----+		+-----+

INNER JOIN Result: Only 1 and 3 (matching in both)

Syntax:

```
SELECT t1.Column1, t2.Column2
FROM Table1 t1
INNER JOIN Table2 t2 ON t1.ForeignKeyColumn = t2.PrimaryKeyColumn;
```

Example 1: Students and Their Courses

```
SELECT
    s.FirstName,
    s.LastName,
    c.CourseName,
    e.Grade
FROM Students s
INNER JOIN Enrollments e ON s.StudentID = e.StudentID
INNER JOIN Courses c ON e.CourseID = c.CourseID;
```

Output:

FirstName	LastName	CourseName	Grade
Raj	Kumar	.NET Full Stack	3.80
Raj	Kumar	Web Development	3.70
Priya	Singh	.NET Full Stack	3.90
Arjun	Patel	Web Development	3.50
Neha	Sharma	Database Design	3.60

Example 2: Courses and Their Instructors

```
SELECT
    i.InstructorName,
    c.CourseName,
    c.Duration
FROM Instructors i
INNER JOIN Courses c ON i.InstructorID = c.InstructorID;
```

Output:

InstructorName	CourseName	Duration
Mr. Sharma	.NET Full Stack	12
Mr. Sharma	Database Design	8
Ms. Priya	Web Development	10

Example 3: Students with Average Grade

```
SELECT
    s.FirstName,
    s.LastName,
```

```

COUNT(e.CourseID) AS CoursesEnrolled,
AVG(e.Grade) AS AverageGrade
FROM Students s
INNER JOIN Enrollments e ON s.StudentID = e.StudentID
GROUP BY s.StudentID, s.FirstName, s.LastName;

```

Output:

FirstName	LastName	CoursesEnrolled	AverageGrade
Raj	Kumar	2	3.75
Priya	Singh	1	3.90
Arjun	Patel	1	3.50
Neha	Sharma	1	3.60

⚠ Key Points:

- Only rows with matches in BOTH tables are returned
 - Students not enrolled in any course won't appear
 - Courses with no enrolled students won't appear
-

← LEFT JOIN (LEFT OUTER JOIN)

What is LEFT JOIN?

LEFT JOIN returns ALL rows from the left table, plus matching rows from the right table.

Visual Representation:

Table A	Table B
+---+---+	+---+---+
1 x	1 a
2 y ←	3 c
3 z	4 d
+---+---+	+---+---+

LEFT JOIN Result: All from A (1,2,3) + matches from B

- 1: matches (1,a)
- 2: no match (NULL for B columns)
- 3: matches (3,c)

Syntax:

```

SELECT t1.Column1, t2.Column2
FROM Table1 t1
LEFT JOIN Table2 t2 ON t1.ForeignKeyColumn = t2.PrimaryKeyColumn;

```

Example 1: All Students with Their Enrollments

```

SELECT
    s.StudentID,
    s.FirstName,
    s.LastName,
    c.CourseName,
    e.Grade
FROM Students s
LEFT JOIN Enrollments e ON s.StudentID = e.StudentID
LEFT JOIN Courses c ON e.CourseID = c.CourseID
ORDER BY s.StudentID;

```

Output:

StudentID	FirstName	LastName	CourseName	Grade
1	Raj	Kumar	.NET Full Stack	3.80
1	Raj	Kumar	Web Development	3.70
2	Priya	Singh	.NET Full Stack	3.90
3	Arjun	Patel	Web Development	3.50
4	Neha	Sharma	Database Design	3.60

Example 2: All Instructors with Their Courses

```

SELECT
    i.InstructorID,
    i.InstructorName,
    c.CourseName,
    c.Credits
FROM Instructors i
LEFT JOIN Courses c ON i.InstructorID = c.InstructorID;

```

Output:

InstructorID	InstructorName	CourseName	Credits
1	Mr. Sharma	.NET Full Stack	4
1	Mr. Sharma	Database Design	3
2	Ms. Priya	Web Development	4
3	Mr. Kumar	NULL	NULL

Note: Mr. Kumar has no courses assigned, so he still appears with NULL values.

Example 3: Find Students NOT Enrolled in Any Course

```

SELECT
    s.StudentID,
    s.FirstName,
    s.LastName
FROM Students s
LEFT JOIN Enrollments e ON s.StudentID = e.StudentID
WHERE e.EnrollmentID IS NULL;

```

Output: Empty (all students are enrolled)

Example 4: Find Instructors WITHOUT Courses

```

SELECT
    i.InstructorID,
    i.InstructorName
FROM Instructors i
LEFT JOIN Courses c ON i.InstructorID = c.InstructorID
WHERE c.CourseID IS NULL;

```

Output:

InstructorID	InstructorName
3	Mr. Kumar

➔ RIGHT JOIN (RIGHT OUTER JOIN)

What is RIGHT JOIN?

RIGHT JOIN returns ALL rows from the right table, plus matching rows from the left table.

Visual Representation:

Table A		Table B
+---+---+		+---+---+
1 x		1 a
2 y	→	3 c
3 z		4 d
+---+---+		+---+---+

RIGHT JOIN Result: All from B (1,3,4) + matches from A

Syntax:

```
SELECT t1.Column1, t2.Column2
FROM Table1 t1
RIGHT JOIN Table2 t2 ON t1.ForeignKeyColumn = t2.PrimaryKeyColumn;
```

Example: All Courses with Their Instructors

```
SELECT
    c.CourseID,
    c.CourseName,
    i.InstructorName
FROM Instructors i
RIGHT JOIN Courses c ON i.InstructorID = c.InstructorID;
```

Output:

CourseID	CourseName	InstructorName
1	.NET Full Stack	Mr. Sharma
2	Web Development	Ms. Priya
3	Database Design	Mr. Sharma

Note:

- RIGHT JOIN is equivalent to LEFT JOIN with tables reversed
- Most use LEFT JOIN, as it's more intuitive

FULL OUTER JOIN

What is FULL OUTER JOIN?

FULL OUTER JOIN returns ALL rows from both tables (matches and non-matches).

Visual Representation:

Table A		Table B
+---+---+		+---+---+
1 x		1 a
2 y	↔	3 c
3 z		4 d
+---+---+		+---+---+

FULL OUTER JOIN: All from both (1,2,3,4)

- 1: matches
- 2: from A only
- 3: matches
- 4: from B only

Syntax:

```
SELECT t1.Column1, t2.Column2
FROM Table1 t1
FULL OUTER JOIN Table2 t2 ON t1.PrimaryKeyColumn =
t2.ForeignKeyColumn;
```

Example:

```
SELECT
    i.InstructorID,
    i.InstructorName,
    c.CourseID,
    c.CourseName
FROM Instructors i
FULL OUTER JOIN Courses c ON i.InstructorID = c.InstructorID
ORDER BY i.InstructorID;
```

Output:

InstructorID	InstructorName	CourseID	CourseName
1	Mr. Sharma	1	.NET Full Stack
1	Mr. Sharma	3	Database Design
2	Ms. Priya	2	Web Development
3	Mr. Kumar	NULL	NULL

✕ CROSS JOIN

What is CROSS JOIN?

CROSS JOIN creates a Cartesian product - combines every row from Table1 with every row from Table2.

Visual Representation:

Table A		Table B
+---+		+---+
1		X
2	✕	Y
3		+---+
+---+		

CROSS JOIN Result:

1-X, 1-Y, 2-X, 2-Y, 3-X, 3-Y (3×2 = 6 rows)

Syntax:

```
SELECT t1.Column, t2.Column
FROM Table1 t1
CROSS JOIN Table2 t2;
```

Example: All Possible Student-Course Combinations

```
SELECT
    s.FirstName,
    s.LastName,
    c.CourseName
FROM Students s
CROSS JOIN Courses c
ORDER BY s.FirstName, c.CourseName;
```

Output (every student with every course):

FirstName	LastName	CourseName
Raj	Kumar	.NET Full Stack
Raj	Kumar	Database Design
Raj	Kumar	Web Development
Priya	Singh	.NET Full Stack

... (all 16 combinations: 4 students × 3 courses)

⚠ Use Cases:

- Creating all possible combinations
 - Generating schedules
 - Calculating permutations
 - Usually used with WHERE clause to filter results
-

Self Join

What is Self Join?

A **Self Join** joins a table to itself to find relationships within the same table.

Example Scenario:

Employees Table:

ID	Name	ManagerID
1	Sharma	NULL
2	Priya	1
3	Kumar	1
4	Neha	2

Example: Find Employee-Manager Relationships

```
CREATE TABLE Employees (  
    EmployeeID INT PRIMARY KEY IDENTITY(1,1),  
    EmployeeName NVARCHAR(100),  
    ManagerID INT,  
    FOREIGN KEY (ManagerID) REFERENCES Employees(EmployeeID)  
);
```

```
INSERT INTO Employees (EmployeeName, ManagerID) VALUES  
( 'Mr. Sharma', NULL),      -- CEO  
( 'Ms. Priya', 1),  
( 'Mr. Kumar', 1),  
( 'Ms. Neha', 2);
```

```
SELECT  
    e.EmployeeName AS Employee,  
    m.EmployeeName AS Manager  
FROM Employees e  
LEFT JOIN Employees m ON e.ManagerID = m.EmployeeID;
```

Output:

Employee	Manager
Mr. Sharma	NULL (CEO, no manager)
Ms. Priya	Mr. Sharma
Mr. Kumar	Mr. Sharma

Multiple Joins

Joining 3+ Tables:

Example: Students, Enrollments, Courses, and Instructors

```
SELECT
    s.FirstName + ' ' + s.LastName AS StudentName,
    c.CourseName,
    i.InstructorName,
    e.Grade,
    c.Duration
FROM Students s
INNER JOIN Enrollments e ON s.StudentID = e.StudentID
INNER JOIN Courses c ON e.CourseID = c.CourseID
INNER JOIN Instructors i ON c.InstructorID = i.InstructorID
WHERE e.Grade > 3.5
ORDER BY s.FirstName, c.CourseName;
```

Output:

StudentName	CourseName	InstructorName	Grade	Duration
Raj Kumar	.NET Full Stack	Mr. Sharma	3.80	12
Raj Kumar	Web Development	Ms. Priya	3.70	10
Priya Singh	.NET Full Stack	Mr. Sharma	3.90	12
Neha Sharma	Database Design	Mr. Sharma	3.60	8

Example: Complex Query with Multiple Conditions

```
SELECT
    s.FirstName,
    s.LastName,
    COUNT(e.CourseID) AS TotalCourses,
    AVG(e.Grade) AS AverageGrade,
    STRING_AGG(c.CourseName, ', ') AS CoursesEnrolled
FROM Students s
LEFT JOIN Enrollments e ON s.StudentID = e.StudentID
LEFT JOIN Courses c ON e.CourseID = c.CourseID
GROUP BY s.StudentID, s.FirstName, s.LastName
HAVING COUNT(e.CourseID) > 0
ORDER BY AverageGrade DESC;
```

Output:

FirstName	LastName	TotalCourses	AverageGrade	CoursesEnrolled
Priya	Singh	1	3.90	.NET Full Stack
Raj	Kumar	2	3.75	.NET Full Stack, Web Development
Neha	Sharma	1	3.60	Database Design
Arjun	Patel	1	3.50	Web Development

Practice Exercises

Setup Tables (Repeat if needed):

```
CREATE TABLE Instructors (
```

```

        InstructorID INT PRIMARY KEY IDENTITY(1,1),
        InstructorName NVARCHAR(100),
        Email VARCHAR(100),
        Department NVARCHAR(50)
    );

CREATE TABLE Courses (
    CourseID INT PRIMARY KEY IDENTITY(1,1),
    CourseName NVARCHAR(100),
    InstructorID INT,
    Credits INT,
    FOREIGN KEY (InstructorID) REFERENCES Instructors(InstructorID)
);

CREATE TABLE Students (
    StudentID INT PRIMARY KEY IDENTITY(1,1),
    FirstName NVARCHAR(50),
    LastName NVARCHAR(50),
    Email VARCHAR(100)
);

CREATE TABLE Enrollments (
    EnrollmentID INT PRIMARY KEY IDENTITY(1,1),
    StudentID INT,
    CourseID INT,
    Grade DECIMAL(3,2),
    FOREIGN KEY (StudentID) REFERENCES Students(StudentID),
    FOREIGN KEY (CourseID) REFERENCES Courses(CourseID)
);

```

Exercise 1: INNER JOIN - Students and Courses

Question: Write a query to show each student's name and the courses they're enrolled in.

► [Click for Answer](#)

Exercise 2: LEFT JOIN - All Students with Enrollment Status

Question: Show all students and their courses, including those not enrolled in any course.

► [Click for Answer](#)

Exercise 3: Filtering with JOIN

Question: Find all students enrolled in courses taught by Mr. Sharma.

► [Click for Answer](#)

Exercise 4: Aggregate with JOIN

Question: Find the average grade for each course.

► [Click for Answer](#)

Exercise 5: Multiple Joins with WHERE

Question: Show students with grades above 3.5 and their instructors.

► [Click for Answer](#)

Exercise 6: Find Unmatched Records

Question: Find all instructors who don't teach any course.

► [Click for Answer](#)

Exercise 7: Self Join (If you have Employees table)

Question: Show each employee and their manager.

► [Click for Answer](#)

Exercise 8: Complex Multi-Join Query

Question: For each student, show: name, courses enrolled, grades, instructors, and average grade.

► [Click for Answer](#)

Interview Q&A

Q1: Difference between INNER JOIN and LEFT JOIN?

Answer: | Aspect | INNER JOIN | LEFT JOIN | |---|-----|-----| | **Matches** | Both tables must match | All left + matching right | | **Null Values** | No NULLs | Right side can have NULLs | | **Rows** | Only matching rows | All left rows | | **Use Case** | Matched data | Including unmatched data |

```
-- INNER JOIN - Only matching
SELECT * FROM Students s
INNER JOIN Enrollments e ON s.StudentID = e.StudentID;
-- Result: Only students with enrollments

-- LEFT JOIN - All students
SELECT * FROM Students s
LEFT JOIN Enrollments e ON s.StudentID = e.StudentID;
-- Result: All students, even those not enrolled
```

Q2: What is a Foreign Key and why is it important?

Answer: - **Foreign Key** is a column that references another table's Primary Key - **Why Important:** - **Data Integrity:** Ensures only valid references - **Relational Structure:** Creates relationships between tables - **Prevents Orphan Records:** Can't have enrollments for non-existent students - **Enforces Constraints:** Database prevents invalid inserts

```
CREATE TABLE Courses (
  CourseID INT PRIMARY KEY,
  InstructorID INT,
  FOREIGN KEY (InstructorID) REFERENCES Instructors(InstructorID)
);

-- This fails - Instructor 99 doesn't exist
```

```
INSERT INTO Courses VALUES (1, 99);

-- This works - Instructor 1 exists
INSERT INTO Courses VALUES (1, 1);
```

Q3: Can you explain 1-to-Many and Many-to-Many relationships?

Answer:

1-to-Many: - One instructor teaches MANY courses - Each course has ONE instructor

```
Instructors (1) → Courses (Many)
1 Sharma      → .NET, Database
1 Priya       → Web Dev
```

Many-to-Many: - Many students take MANY courses - Many courses have MANY students - Requires a junction table (Enrollments)

```
Students (Many) ↔ Courses (Many)
                  (via Enrollments)

Raj      ← → .NET
Priya    ← → Web Dev
Arjun    ← → Database
```

Q4: What is a junction/bridge table?

Answer: A **junction table** (bridge table) stores the relationship between two many-to-many tables.

```
-- Many-to-Many: Students ↔ Courses
Students (1) → Enrollments (Many) ← Courses (1)

-- Enrollments is the junction table
CREATE TABLE Enrollments (
    EnrollmentID INT PRIMARY KEY,
    StudentID INT,          -- FK to Students
    CourseID INT,           -- FK to Courses
    Grade DECIMAL(3,2),
    FOREIGN KEY (StudentID) REFERENCES Students(StudentID),
    FOREIGN KEY (CourseID) REFERENCES Courses(CourseID)
);
```

Q5: What happens if you DELETE a record referenced by a Foreign Key?

Answer: It depends on the **Delete Constraint**:

```
-- Option 1: CASCADE - Delete child records automatically
FOREIGN KEY (InstructorID) REFERENCES Instructors(InstructorID)
ON DELETE CASCADE; -- If instructor deleted, their courses deleted too

-- Option 2: RESTRICT - Prevent deletion
ON DELETE RESTRICT; -- Can't delete instructor if they teach courses

-- Option 3: SET NULL - Set foreign key to NULL
ON DELETE SET NULL; -- Course's instructor becomes NULL
```

Example:

```
-- If Ms. Priya (InstructorID 2) is deleted:
-- Courses with InstructorID = 2 will also be deleted (if CASCADE)
DELETE FROM Instructors WHERE InstructorID = 2;
```

Q6: Can you write a query to find students who took MORE than one course?

Answer:

```
SELECT
    s.FirstName,
    s.LastName,
    COUNT(e.CourseID) AS CourseCount
FROM Students s
INNER JOIN Enrollments e ON s.StudentID = e.StudentID
GROUP BY s.StudentID, s.FirstName, s.LastName
HAVING COUNT(e.CourseID) > 1;
```

Q7: What is the difference between WHERE and JOIN conditions?

Answer: | Aspect | WHERE | JOIN Condition | |----|----|-----| |
Purpose | Filter rows | Link tables | | **Timing** | After JOIN | During JOIN | | **Order** | After JOIN in execution | Before WHERE |

```
SELECT * FROM Students s
INNER JOIN Enrollments e ON s.StudentID = e.StudentID -- JOIN
condition
WHERE e.Grade > 3.5; -- WHERE condition (filters after join)
```

Q8: How do you avoid cartesian products in JOINS?

Answer: Always specify the JOIN condition!

```
-- WRONG - Cartesian product! (4 students x 5 courses = 20 rows)
SELECT * FROM Students, Enrollments;

-- CORRECT - Specify JOIN condition
SELECT * FROM Students s
INNER JOIN Enrollments e ON s.StudentID = e.StudentID;
```

Q9: What is the difference between INNER JOIN and CROSS JOIN?

Answer: | Aspect | INNER JOIN | CROSS JOIN | |----|-----|
-----| | **Condition** | Requires ON condition | No condition | |
Result | Matching rows | All combinations | | **Usage** | Link related data | Generate combinations |

```
-- INNER JOIN - Only matching
SELECT * FROM Students s
INNER JOIN Enrollments e ON s.StudentID = e.StudentID;

-- CROSS JOIN - All combinations (4 x 3 = 12)
SELECT * FROM Students s
CROSS JOIN Courses c;
```

Q10: Can you explain table aliases and why they're useful?

Answer: **Table aliases** are short names for tables, especially useful for: - **Readability:** Shorter code - **Self Joins:** Distinguish copies of same table - **Clarity:** Know which table a column comes from

```
-- Without alias (hard to read)
SELECT Students.FirstName, Enrollments.Grade
FROM Students INNER JOIN Enrollments
ON Students.StudentID = Enrollments.StudentID;

-- With alias (clearer)
SELECT s.FirstName, e.Grade
FROM Students s
INNER JOIN Enrollments e ON s.StudentID = e.StudentID;

-- Essential for Self Joins
SELECT e.EmployeeName, m.EmployeeName
FROM Employees e
LEFT JOIN Employees m ON e.ManagerID = m.EmployeeID;
```

Summary: JOIN Comparison Table

JOIN Type	Returns	Example
INNER	Only matching rows	3 matches
LEFT	All left + matching right	4 rows (all left)
RIGHT	All right + matching left	5 rows (all right)
FULL	All rows from both	6 rows
CROSS	All combinations	20 rows (4×5)
SELF	Table with itself	Hierarchies

Key Takeaways

1. **Foreign Keys enforce relationships** between tables
 2. **INNER JOIN** for matching data only
 3. **LEFT JOIN** to include unmatched left records
 4. **Always use ON clause** to avoid cartesian products
 5. **Table aliases** make code cleaner
 6. **Junction tables** handle many-to-many relationships
 7. **Test with SELECT** before DELETE operations with FKs
-

Next Steps

- Practice all 8 exercises multiple times
- Create your own schema with relationships
- Understand normalization (1NF, 2NF, 3NF)
- Move to advanced queries (GROUP BY, aggregate functions)

Happy Coding!